

# tp intro javascript

Matisse  
kra  
bts sio  
Sommaire

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>objectif</b>                                        | <b>2</b>  |
| <b>Conception et réalisation</b>                       | <b>2</b>  |
| Étape 1 : Environnement de travail et "Hello World"    | 2         |
| Étape 2 : Les Variables                                | 2         |
| Étape 3: Les Types de données et Conversions           | 4         |
| Étape 4 : Interactions (Alert, Prompt, Confirm)        | 6         |
| Étape 5: Les Opérateurs mathématiques et d'Affectation | 6         |
| Quels sont les résultats de ces expressions ?          | 8         |
| étape 6 les comparaison                                | 9         |
| Étape 6 : Conditions (If/Else)                         | 9         |
| etape 7 opérateur logique :                            | 12        |
| Étape 8: Les Boucles (Loops)                           | 17        |
| Étape 9 : L'instruction Switch                         | 20        |
| Étape 10 : Les Fonctions                               | 22        |
| <b>Conclusion</b>                                      | <b>26</b> |

## objectif

L'objectif de cet atelier est de découvrir le langage **JavaScript**, qui permet de rendre les pages web dynamiques et interactives, contrairement au HTML/CSS qui sont statiques. Le but est de comprendre la syntaxe de base, comment lier un script à une page HTML, comment stocker des données (variables) et comment créer une logique conditionnelle (si... alors...).

## Conception et réalisation

La progression de l'atelier s'est décomposée en plusieurs étapes pour maîtriser les fondamentaux.

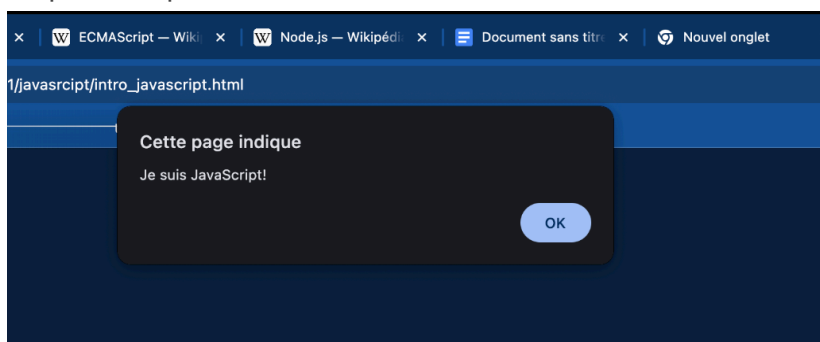
### Étape 1 : Environnement de travail et "Hello World"

Dans un premier temps, j'ai appris à intégrer du code JavaScript dans une page HTML.

- **Balise script** : J'ai utilisé la balise `<script>` pour insérer le code directement dans le HTML
- **Console** : J'ai découvert la console du navigateur pour tester des commandes et voir les erreurs

exercice réalisé:

- **exo1**: Créez une page qui affiche un message "Je suis JavaScript!". Assurez-vous simplement que cela fonctionne.



### Étape 2 : Les Variables

J'ai appris à stocker des données. J'ai utilisé le mot-clé `let` pour déclarer des variables qui peuvent changer, et `const` pour des valeurs fixes.

#### Exercice réalisé :

exo1 Travailler avec des variables:

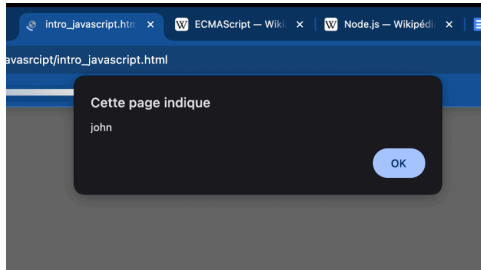
Déclarez deux variables : `admin` and `name`.

1. Assignez la valeur "John" à `name`.
2. Copiez la valeur de `name` à `admin`.
3. Affichez la valeur de `admin` en utilisant `alert` (devrait afficher "John").

Matisse kra  
bts sio

4.

J'ai déclaré deux variables `admin` et `name`. J'ai assigné "John" à `name`, puis j'ai copié la valeur dans `admin` pour l'afficher via une alerte.



```
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <script>
5        let admin;
6        let name = "john";
7        admin = name;
8        alert(admin);
9      </script>
10   </body>
11 </html>
```

## exo 2 Assigner le bon nom

1. Créez la variable avec le nom de notre planète. Comment nommeriez-vous une telle variable ?
2. Créez la variable pour stocker le nom du visiteur actuel. Comment nommeriez-vous cette variable ?

```
<!DOCTYPE html>
<html>
  <body>
    <script>
      let planete_terre;
      let currentUser;
    </script>
  </body>
</html>
```

### exo 3 Constante en majuscule ?

Examinez le code suivant :

```
1 const birthday = '18.04.1982';  
2  
3 const age = someCode(birthday);
```

Ici, nous avons une constante `birthday` pour la date de naissance, ainsi que la constante `age`.

L'`age` est calculé à partir de `birthday` en utilisant `someCode()`, ce qui signifie un appel de fonction que nous n'avons pas encore expliqué (nous le ferons bientôt !), mais les détails n'ont pas d'importance ici, le fait est que l'`age` est calculé d'une manière ou d'une autre en fonction de la date de naissance `birthday`.

Serait-il juste d'utiliser des majuscules pour `birthday` ? Pour `age` ? Ou même pour les deux ?

```
1 const BIRTHDAY = '18.04.1982'; // mettre l'anniversaire en majuscule ?  
2  
3 const AGE = someCode(BIRTHDAY); // mettre l'âge en majuscule ?
```

```
1 <!DOCTYPE html>  
2 <html>  
3   <body>  
4     <script>  
5       const birthday = '18.04.1982';  
6       const age = someCode(birthday);  
7       /*mettre age et birthday en minuscule car ce ne sont  
8        pas des valeur coder en dur (hexadecimal ou autre)*/  
9     </script>  
10  </body>  
11 </html>
```

### Étape 3: Les Types de données et Conversions

J'ai exploré les différents types de valeurs que JavaScript permet de manipuler. Contrairement à certains langages stricts, JavaScript est dynamique, mais il possède des types bien définis :

- **Number** : Pour les nombres entiers et à virgule. J'ai noté qu'il existe une limite de sécurité pour les entiers ( $2^{53}-1$ ).
- **BigInt** : Un type ajouté récemment pour gérer les entiers arbitrairement grands, utile pour la cryptographie ou les timestamps précis.
- **Boolean** : Le type logique qui ne possède que deux valeurs : `true` (vrai) et `false` (faux), essentiel pour les conditions.
- **Null** : Une valeur spéciale qui ne représente "rien", "vide" ou une "valeur inconnue". J'ai bien compris que ce n'est pas une erreur, mais une valeur assignée volontairement.
- **Undefined** : Le type par défaut d'une variable déclarée mais non initialisée.

Les Conversions de types J'ai appris qu'il est souvent nécessaire de convertir une donnée d'un type à l'autre (par exemple, ce que l'utilisateur tape est toujours une `String`).

- J'ai utilisé des fonctions comme `Number(valeur)`, `String(valeur)` et `Boolean(valeur)` pour effectuer ces transformations explicites.
- J'ai retenu une règle importante : lors d'une conversion en nombre, `null` devient `0`, tandis que `undefined` devient `NaN` (Not a Number), ce qui peut causer des erreurs de calcul.

exercice réaliser:

exercice 4:String quotes

Quelle est la sortie du script ?

Matisse kra  
bts sio

```
1 let name = "Ilya";  
2  
3 alert( `hello ${1}` ); // ?  
4  
5 alert( `hello ${"name"}` ); // ?  
6  
7 alert( `hello ${name}` ); // ?
```

réponse :

```
<!DOCTYPE html>  
✓<html>  
✓  
✓<body>  
  <script>  
    let name ="Ilya";  
    alert( `hello${1}` );//sortie hello1  
    alert( `hello${"name"}` );//sortie helloname  
    alert( `hello ${name}` );//sortie hello Ilya  
  </script>  
</body>  
</html>
```

Cette page indique

hello1

OK

Cette page indique

helloname

Cette page indique

hello Ilya

## Étape 4 : Interactions (Alert, Prompt, Confirm)

Pour interagir avec le visiteur, j'ai utilisé trois fonctions natives du navigateur :

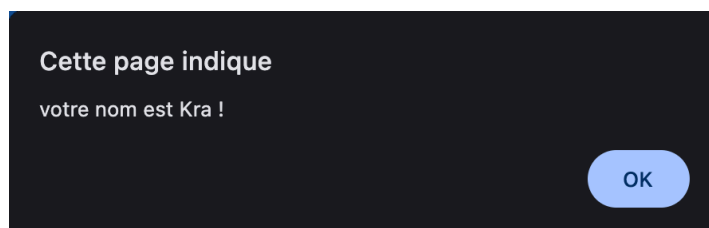
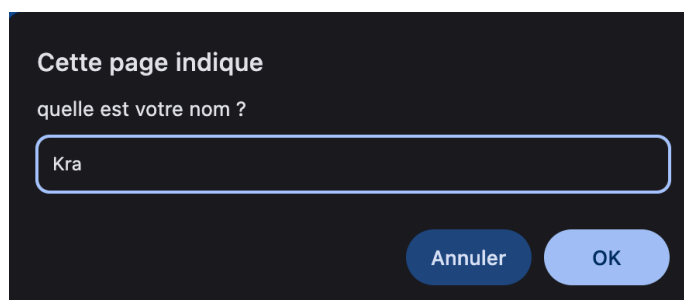
- `alert()` pour afficher un message.
- `prompt()` pour demander une saisie à l'utilisateur (comme son nom ou un nombre).
- `confirm()` pour demander une validation (OK/Annuler).

exercice réaliser:

exercice 5: Une simple page

Créez une page Web qui demande un nom et l'affiche

```
1 <!DOCTYPE html>
2 <html>
3   <body>
4     <script>
5       let name = prompt("quelle est votre nom ?");
6       alert( `votre nom est ${name} !` );
7     </script>
8   </body>
9 </html>
```



## Étape 5: Les Opérateurs mathématiques et d'Affectation

J'ai manipulé les opérateurs pour effectuer des calculs et modifier la valeur de mes variables.

- Opérateurs arithmétiques de base : J'ai utilisé les signes classiques : addition `+`, soustraction `-`, multiplication `*` et division `/`.
- Le Modulo (`%`) : J'ai découvert que ce signe ne sert pas à faire des pourcentages, mais à obtenir le reste d'une division entière.
  - Exemple : `5 % 2` donne `1`.

- L'Exposant(\*\*) : C'est un opérateur récent qui permet de calculer une puissance (ex: 23 s'écrit `2 ** 3`).
- L'Affectation et la modification :
  - J'ai compris que le signe `=` retourne une valeur, ce qui permet des enchaînements (ex: `a = b = c`).
  - J'ai utilisé des raccourcis pour modifier une variable "sur place", comme `+=` ou `*=` (ex: `n += 5` est identique à `n = n + 5`).
- Incrémentation (`++`) et Décrémentement (`--`) : Ces opérateurs très courants permettent d'augmenter ou diminuer une variable de 1 .
  - *Point de vigilance* : J'ai noté que cela ne s'applique qu'aux variables. Faire `5++` provoquerait une erreur.

L'opérateur `+` binaire J'ai remarqué une particularité importante : l'opérateur `+` sert à la fois à l'addition (nombres) et à la concaténation (chaînes). Si l'un des opérandes est une chaîne de caractères, JavaScript convertit l'autre en chaîne et les colle ensemble.

exercice realiser:

#### exercice 5 Les formes postfixes et préfixes

Quelles sont les valeurs finales de toutes les variables `a`, `b`, `c` et `d` après le code ci-dessous ?

```
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <script>
5        let a = 1, b = 1;
6        let c = ++a; // a passe de 1 à 2 la valeur de c est 2
7        let d = b++; // la valeur de b passe de 1 à 2 la valeur de d est 1
8      </script>
9    </body>
10 </html>
```

#### exo 6 Résultat d'affectation

Quelles sont les valeurs de `a` et `x` après le code ci-dessous ?

```
1  <script>
2    let a = 2;
3    let x = 1 + (a *= 2);
4    // a vaut 4 et x vaut 5
5  </script>
```

L'opérateur `*=` est un opérateur d'affectation. Il multiplie la valeur actuelle de `a` par 2 et met à jour `a` immédiatement.

Matisse kra  
bts sio

## exo 7 Les conversions de type

Quels sont les résultats de ces expressions ?

```
1 <!DOCTYPE html>
2 <html>
3   <body>
4     <script>
5       "" + 1 + 0 // "10"
6       "" - 1 + 0 // -1
7       true + false // 1
8       6 / "3" // 2
9       "2" * "3" // 6
10      4 + 5 + "px" // "9px"
11      "$" + 4 + 5 // "$45"
12      "4" - 2 // 2
13      "4px" - 2 // NaN
14      "-9" + 5 // "-95"
15      "-9" - 5 // -14
16      null + 1 // 1
17      undefined + 1 // NaN
18      " \t \n " - 2 // -2
19    </script>
20  </body>
21 </html>
```

## exo 8 Corrigez l'addition

Voici un code qui demande à l'utilisateur deux nombres et affiche leur somme.

Cela ne fonctionne pas correctement. La sortie dans l'exemple ci-dessous est 12 (pour les valeurs d'invite par défaut).

Pourquoi ? Réparez-le. Le résultat doit être 3.

```
1 let a = prompt("First number?", 1);
2 let b = prompt("Second number?", 2);
3
4 alert(a + b); // 12
```

1. Pourquoi cela affiche "12" ?

La fonction prompt renvoie toujours le résultat sous forme de chaîne de caractères (String), même si l'utilisateur tape un chiffre. Pour l'ordinateur, `a` vaut "1" (texte) et `b` vaut "2" (texte). En JavaScript, quand on utilise le signe `+` entre deux chaînes de texte, il fait une concaténation au lieu d'une addition mathématique. Donc : "1" + "2" = "12".

2. solution

```
1 <script>
2   let a = prompt("Entrez un nombre", 1);
3   let b = prompt("Entrez un autre nombre", 2);
4   alert(+a + +b);
5 </script>
```

Il faut convertir les chaînes de caractères en nombres avant de faire l'addition. ce que fait le `“+”` avant les variable `a` et `b`



## étape 6 les comparaison

J'ai étudié comment comparer des valeurs entre elles. J'ai compris que le résultat d'une comparaison est toujours une valeur booléenne : soit true (vrai), soit false (faux).

- Opérateurs classiques : J'ai utilisé les signes mathématiques standards : > (supérieur), < (inférieur), >= (supérieur ou égal) et <= (inférieur ou égal).
- L'Égalité : J'ai appris à faire la distinction entre deux types d'égalités :
  - L'égalité simple == : qui vérifie la valeur mais peut convertir les types automatiquement (ce qui peut parfois créer des erreurs).
  - L'égalité stricte === : qui vérifie à la fois la valeur et le type. C'est celle qui est recommandée pour éviter les bugs.
  - Exemple : 0 == false est vrai (car 0 équivaut à faux), mais 0 === false est faux (car l'un est un nombre et l'autre un booléen).
- Comparaison de chaînes : J'ai noté que les chaînes de caractères sont comparées lettre par lettre selon l'ordre lexicographique (l'ordre du dictionnaire).

exercice réaliser :

exercice 9 Comparaisons

Quel sera le résultat pour les expressions suivantes :

```
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <script>
5        5 > 4 // true
6        "apple" > "pineapple" //false
7        "2">"12" // true
8        undefined == null // true
9        undefined === null // false
10       null == "\n0\n" // false
11       null === + "\n0\n" // false
12     </script>
13   </body>
14 </html>
```

## Étape 6 : Conditions (If/Else)

C'est la partie algorithmique. J'ai utilisé les instructions `if` et `else` pour exécuter du code seulement si certaines conditions sont vraies. J'ai aussi manipulé les opérateurs de comparaison (comme `===`, `>`, `||`, `&&`).

Matisse kra  
bts sio

exercice réaliser :

exercice 10 if (une chaîne de caractères avec zéro)

Est-ce que `alert` sera affiché ?

```
1 if ("0") {  
2   alert( 'Hello' );  
3 }
```

L'alerte s'affiche parce que la chaîne de caractères "0" est considérée comme Vraie (true). Dans une condition `if (...)`, JavaScript convertit automatiquement la valeur entre parenthèses en un booléen.

1. Le Nombre 0 : En programmation, le nombre zéro représente souvent l'absence de valeur ou le "Faux".
  - `if (0)` → C'est Faux (pas d'alerte).
2. La Chaîne "0" : Ici, vous avez des guillemets. Pour JavaScript, ce n'est pas un nombre, c'est du texte. Et n'importe quel texte qui n'est pas vide est considéré comme Vrai.
  - `if ("0")` → C'est Vrai (alerte !)

exercice 11 Le nom de JavaScript

En utilisant la construction `if...else`, écrivez le code qui demande : 'Quel est le nom "officiel" de JavaScript?' Si le visiteur entre "ECMAScript", alors éditez une sortie "Bonne réponse !", Sinon – retourne "Ne sait pas ? ECMAScript!"

```
Volumes > LaCie > bts sio 1 > javascript > <> intro_javascript.html > html  
1 <!DOCTYPE html>  
2 <html>  
3   <body>  
4     <script>  
5       let reponse = prompt('Quel est le nom "officiel" de JavaScript ?');  
6  
7       if (reponse === "ECMAScript") {  
8         alert("Bonne réponse !");  
9       } else {  
10        alert("Ne sait pas ? ECMAScript !");  
11      }  
12    </script>  
13  </body>  
14 </html>
```

Cette page indique  
quel est le nom de javascript ?

Annuler OK

Cette page indique  
ne sait pas ? ecmaascript !

OK

Matisse kra  
bts sio

### exercice 12 Afficher le signe

En utilisant if..else, écrivez le code qui obtient un numéro via le prompt, puis l'affiche en alert :

- 1, si la valeur est supérieure à zéro,
- -1, si inférieur à zéro,
- 0, si est égal à zéro.

Dans cet exercice, nous supposons que l'entrée est toujours un nombre.

```
1 <!DOCTYPE html>
2 <html>
3   <body>
4     <script>
5       let nombre = +prompt("Entrez un nombre :");
6
7       if (nombre > 0) {
8         alert(1);
9       } else if (nombre < 0) {
10        alert(-1);
11      } else {
12        alert(0);
13      }
14    </script>
15  </body>
16 </html>
```

Cette page indique

Entrez un nombre :

Annuler OK

Cette page indique

1

OK

### exercice 13 Réécrire 'if' en '?'

Réécrivez ce if en utilisant l'opérateur conditionnel '?' :

```
1 let result;
2
3 if (a + b < 4) {
4   result = 'Below';
5 } else {
6   result = 'Over';
7 }
```

```
1 <!DOCTYPE html>
2 <html>
3   <body>
4     <script>
5       let result = (a + b < 4) ? 'below' : 'over';
6     </script>
7   </body>
8 </html>
```

Matisse kra  
bts sio

exercice 14 Réécrire 'if..else' en '?'

Réécrire ce if..else en utilisant plusieurs opérateurs ternaires '?'.  
Pour plus de lisibilité, il est recommandé de diviser le code en plusieurs lignes.

```
1 let message;
2
3 if (login == 'Employee') {
4   message = 'Hello';
5 } else if (login == 'Director') {
6   message = 'Greetings';
7 } else if (login == '') {
8   message = 'No login';
9 } else {
10  message = '';
11 }
```

```
1 <script>
2   let message = (login == 'employee') ? 'hello' :
3                 (login == 'director') ? 'greetings' :
4                 (login == '') ? 'no login' :
5                 '';
6 </script>
```

#### etape 7 opérateur logique :

Pour affiner mes conditions et combiner plusieurs vérifications en même temps, j'ai utilisé les opérateurs logiques. J'ai retenu qu'il en existe trois principaux en JavaScript :

- OU (||) : J'ai appris qu'il renvoie true si au moins une des conditions est vraie.
  - Fonctionnement particulier : J'ai découvert qu'en JavaScript, le OU s'arrête dès qu'il trouve une valeur vraie et la renvoie directement (on appelle ça le court-circuit).
- ET (&&) : Il renvoie true uniquement si toutes les conditions sont vraies. S'il rencontre une seule fausseté, il s'arrête et renvoie faux.
- NON (!) : Cet opérateur inverse simplement la valeur de vérité (vrai devient faux et inversement).

exercice realiser

exercice 15 Quel est le résultat de OR ?

Qu'est-ce que le code ci-dessous va sortir ?

```
1 <script>
2   alert(null||2||undefined);// affiche 2 car c'est la première valeur "vraie"
3 </script>
```

Matisse kra  
bts sio

exercice 16 Quel est le résultat des alertes OR ?  
Qu'est-ce que le code ci-dessous va sortir ?

```
1  <script>
2  alert(alert(1) || 2 || alert(3)); /* affiche 1 puis 2
3  comme alert(1) retourne undefined valeur fausse,
4  l'opérateur || évalue la deuxième opérande
5  qui est 2 valeur vrai et s'arrête là.
6  */
7  </script>
```

exercice 17 Quel est le résultat de AND ?  
Qu'est-ce que ce code va afficher ?

```
1  script>
2  alert(1 && null && 2); //affiche null car c'est la première valeur fausse
3  /script>
```

exercice 18 Quel est le résultat des alertes AND ?  
Qu'est-ce que ce code va afficher ?

```
1  <script>
2  alert(alert(1) && alert(2)); //affiche 1 puis undefined
3  </script>
```

JavaScript doit évaluer ce qu'il y a à l'intérieur des parenthèses principales avant d'exécuter le dernier alert. Il regarde donc : alert(1) && alert(2)

L'opérateur && fonctionne de gauche à droite avec une règle appelée "Court-circuit" (Short-circuit evaluation) :

1. Il exécute la partie gauche : alert(1).
  - Résultat visible : La fenêtre affiche "1".
2. Il récupère le retour de cette fonction : undefined.
3. En booléen, undefined est considéré comme Faux.
4. Le court-circuit : Comme la première condition est fausse, l'opérateur && sait que le résultat total sera forcément faux. Il n'exécute donc pas la deuxième partie (alert(2)).
  - C'est pour cela que "2" ne s'affiche jamais.

exercice 19 Le résultat de OR AND OR  
Quel sera le résultat ?

```
1  <script>
2  alert(null || 1 && 3 || 4);
3  </script>
```

## 1. Priorité

Matisse kra  
bts sio

À cause de la priorité du `&&`, JavaScript ne lit pas strictement de gauche à droite. Il "groupe" d'abord le ET. Il voit ton code comme ceci :

`null || (1 && 3) || 4`

## 2. L'exécution étape par étape

1. Calcul du groupe prioritaire (`1 && 3`) :
  - 1 est "Vrai" (truthy).
  - Dans un ET, si le premier est vrai, on renvoie le dernier élément vérifié.
  - Résultat intermédiaire : 3.
2. L'expression devient : `null || 3 || 4`
3. Calcul du premier OU (`null || 3`) :
  - `null` est "Faux" (falsy).
  - Dans un OU, si le premier est faux, on passe au suivant.
  - Résultat intermédiaire : 3.
4. Calcul du dernier OU (`3 || 4`) :
  - L'expression restante est `3 || 4`.
  - 3 est "Vrai".
  - Dans un OU, dès qu'on trouve un Vrai, on s'arrête (Court-circuit). On renvoie la valeur immédiatement.
  - Résultat final : 3.

exercice 19 Vérifiez la plage entre

Écrivez une condition "if" pour vérifier que l'âge est compris entre 14 et 90 ans inclus.

"Inclus" signifie que l'âge peut atteindre les 14 ou 90 ans.

```
1  <script>
2  let age = prompt("saisissez votre âge ?");
3  if ( age <= 90 && age >= 14 ){
4  |   alert("Votre est entre 14 et 90 ans");
5  } else {
6  |   alert("Votre âge n'est pas entre 14 et 90 ans");
7  }
8  </script> |
```

exercice 20

Ecrivez une condition `if` pour vérifier que l'âge n'est PAS compris entre 14 et 90 ans inclus

Créez deux variantes: la première utilisant `NOT` !, La seconde – sans ce dernier.

```
1  <script>
2  let age = Number(prompt("Saisissez votre âge ?"));
3  if ( ! (age >= 14 && age <= 90) ) {
4  |   alert("Votre âge n'est pas entre 14 et 90 ans");
5  } else {
6  |   alert("Votre âge est entre 14 et 90 ans");
7  }
8  </script>
```

```
1  <script>
2  let age = prompt("saissisez votre âge ?");
3  if ( age >= 90 || age <= 14 ){
4      alert("Votre âge n'est pas entre 14 et 90 ans");
5  } else {
6      alert("Votre âge est entre 14 et 90 ans");
7  }
8  </script>
```

exercice 21 Une question à propos de "if"

Lesquelles de ces alertes vont s'exécuter ?

Quels seront les résultats des expressions à l'intérieur de if (...) ?

```
1  <script>
2  if (-1 || 0) alert('first');//affiche car -1 est vrai
3  if (-1 && 0) alert('second');//ne s'affiche pas car 0 est faux
4  if (null || -1 && 1) alert('third');//affiche car -1 && 1 est vrai et donc null || vrai est vrai
5  </script>
```

exercice 22 Check the login

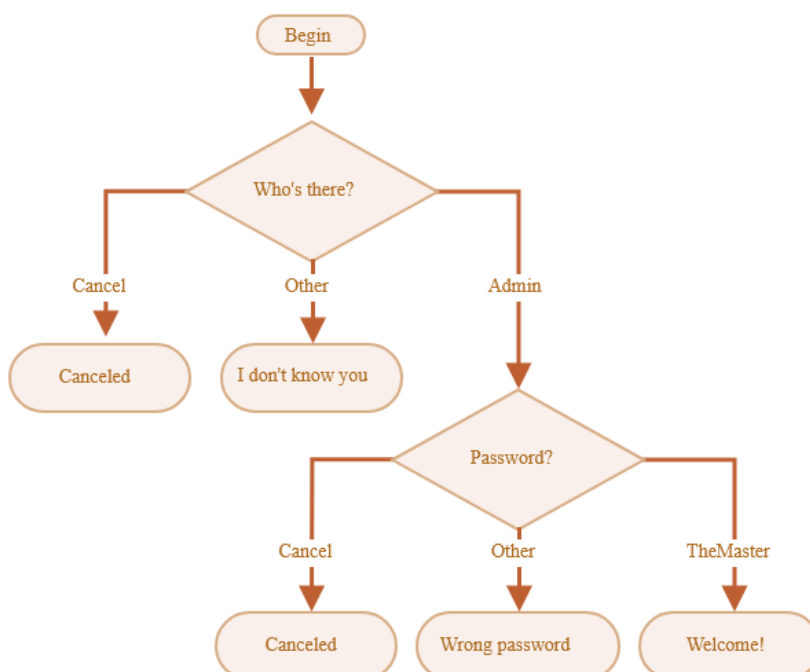
Écrivez le code qui demande une connexion avec prompt.

Si le visiteur entre "Admin", puis prompt pour un mot de passe, si l'entrée est une ligne vide ou Esc – affichez "Canceled", s'il s'agit d'une autre chaîne de caractères – alors affichez "I don't know you".

Le mot de passe est vérifié comme suit :

- S'il est égal à "TheMaster", alors affichez "Welcome!",
- Une autre chaîne de caractères – affichez "Wrong password",
- Pour une chaîne de caractères vide ou une entrée annulée, affichez "Canceled".

Le schéma :



Matisse kra  
bts sio

```
1  <script>
2  let bienvenue = "qui est-ce ?";
3  alert("Bonjour " + bienvenue);
4  let user = prompt ("annulez, autre ou admin ?");
5  if (user === "admin") {
6      let password = prompt("mot de passe ?");
7      if (password === "TheMaster") {
8          alert("Bienvenue !");
9      } else if (password === null || password === " ") {
10         alert("cancelld");
11     } else {
12         alert("Mot de passe incorrect");
13     }
14 } else if (user === null) {
15     alert("Annulé");
16 } else {
17     alert("Je ne connais pas cet utilisateur");
18 }
19
20 </script>
```

Cette page indique

Bonjour qui est-ce ?

OK

Cette page indique

annulez, autre ou admin ?

admin

Annuler

OK

Cette page indique

mot de passe ?

|

Annuler

OK

Cette page indique

Bienvenue !

OK



## Étape 8: Les Boucles (Loops)

Pour éviter de répéter du code manuellement (comme afficher 10 fois un message), j'ai appris à utiliser les boucles qui répètent une action tant qu'une condition est vraie.

- La boucle while : C'est la plus simple, elle tourne tant que la condition est vérifiée.
- La boucle do...while : J'ai noté sa particularité : elle exécute le code au moins une fois avant de vérifier la condition.
- La boucle for : C'est la plus utilisée. J'ai bien compris sa structure en trois parties : (début; condition; étape).
  - Exemple : for (let i = 0; i < 3; i++) permet de compter de 0 à 2.
- Contrôle de boucle : J'ai découvert deux mots-clés essentiels :
  - break : pour arrêter immédiatement la boucle (par exemple si l'utilisateur annule une saisie).
  - continue : pour sauter l'itération en cours et passer directement à la suivante (pratique pour ignorer certaines valeurs, comme les nombres impairs).

exercice realiser

exo 23: Dernière valeur de boucle

```
Volumes > LaCie > bts sio 1 > javascript > <> intro_javascript.html > script
1  <script>
2  let i = 3;
3  while (i) {
4      alert(i--); // 3, 2, 1 car i décrémente de 1 après chaque alerte jusqu'à ce que i soit 0
5  }
6  </script>
```

exo 24 : Quelles valeurs affiche la boucle while ?

A votre avis, quelles sont les valeurs affichées pour chaque boucle ? Notez-les puis comparez avec la réponse.

Les deux boucles affichent-elles les mêmes valeurs dans l'alert ou pas ?

1.

Le préfixe sous forme ++i :

```
1  let i = 0;
2  while (++i < 5) alert( i );
```

2.

Le postfixe sous forme i++ :

```
1  let i = 0;
2  while (i++ < 5) alert( i );
```

```
1  <script>
2  let i = 0;
3  while (i++ < 5 ) alert(i--); // 1,2,3,4,5 i est incrémenté jusqu'à 6 avant l'alerte et décrémente après
4  </script>
```

```
1  <script>
2  let i = 0;
3  while (++i < 5 ) alert(i--); // 1 2 3 4 i est incrémenté avant la comparaison et décrémente après l'alerte
4  </script>
```

### exo 25: Quelles valeurs sont affichées par la boucle "for" ?

Pour chaque boucle, notez les valeurs qui vont s'afficher. Ensuite, comparez avec la réponse.

Les deux boucles `alert` les mêmes valeurs ou pas ?

1.

La forme postfix :

```
1 for (let i = 0; i < 5; i++) alert( i );
```

2.

La forme préfix :

```
1 for (let i = 0; i < 5; ++i) alert( i );
```

```
1 <script>
2 for (let i = 0; i < 5; i++) alert(i); // i est incrémenté jusqu'à 4
3 </script>
```

```
1 <script>
2 for (let i = 0; i < 5; ++i) alert(i); // 0,1,2,3,4
3 </script>
```

### exo 26 : Extraire les nombres pairs dans la boucle

Utilisez la boucle `for` pour afficher les nombres pairs de 2 à 10.

```
<script>
  // On parcourt tout de 0 à 10
  for (let i = 0; i < 10; i++) {
    // On teste SI le nombre est impair
    if (i % 2 == 1) {
      // On passe à l'itération suivante
      continue;
    }
    alert(i);
  }
</script>
```

Matisse kra  
bts sio

### exo 27 : Remplacer "for" par "while"

Réécrivez le code en modifiant la boucle `for` en `while` sans modifier son comportement (la sortie doit rester la même).

```
1 for (let i = 0; i < 3; i++) {  
2   alert( `number ${i}!` );  
3 }
```

```
1  <script>  
2    let i = 0;  
3    Click to collapse the range.  
4    alert(`number ${i}!`);  
5    i++;  
6  }  
7  </script>
```

### exo 28 : Répéter jusqu'à ce que l'entrée soit correcte

Ecrivez une boucle qui demande un nombre supérieur à `100`. Si le visiteur saisit un autre numéro, demandez-lui de le saisir à nouveau.

La boucle doit demander un numéro jusqu'à ce que le visiteur saisisse un nombre supérieur à `100` ou annule l'entrée/entre une ligne vide.

Ici, nous pouvons supposer que le visiteur ne saisit que des chiffres. Il n'est pas nécessaire de mettre en œuvre un traitement spécial pour une entrée non numérique dans cette tâche.

```
1  <script>  
2    let nombre; // 1. Déclaration AVANT la boucle  
3  
4    do {  
5      nombre = prompt("Entrer un nombre supérieur à 100:");  
6  
7      // Sécurité : Si l'utilisateur clique sur Annuler, nombre sera NaN (Not a Number) ou null  
8      if (!nombre && nombre !== 0) break;  
9  
10   } while (nombre <= 100); // 2. On boucle TANT QUE c'est faux (inférieur ou égal à 100)  
11  
12   // Si on est ici, c'est que la boucle est finie  
13   alert("Félicitations! " + nombre + " est bien supérieur à 100.");  
14 </script>
```

Matisse kra  
bts sio

exo 29 : Extraire des nombres premiers

Écrivez un code qui produit les nombres premiers dans l'intervalle 2 à n.

Pour n = 10, le résultat sera 2,3,5,7.

P.S. Le code devrait fonctionner pour n'importe quel n et aucune valeur fixe ne doit être codé en dur.

```
Volumes > LaCie > bts sio 1 > javascript > <> intro_javascript.html > script
1  <script>
2  // 1. On convertit la saisie en Entier pour être propre
3  let nombre = parseInt(prompt("Jusqu'à quel nombre veux-tu les nombres premiers ?"));
4
5  // Variable pour stocker le résultat final
6  let resultat = "";
7
8  // 2. On commence à 2 (car l'énoncé dit n > 1)
9  for (let i = 2; i <= nombre; i++) {
10
11     // --- Initialisation du Drapeau ---
12     // On suppose au départ que i EST premier (Vrai)
13     let estPremier = true;
14
15     // 3. Boucle de vérification
16     for (let j = 2; j < i; j++) {
17         // Si i est divisible par j (le reste est 0)
18         if (i % j == 0) {
19             estPremier = false; // On baisse le drapeau : ce n'est PAS un premier
20             break; // IMPORTANT : On arrête de chercher (break), on sait déjà que c'est perdu
21         }
22     }
23
24     // 4. Décision d'affichage
25     // On n'affiche QUE si le drapeau est resté Vrai
26     if (estPremier == true) {
27         resultat = resultat + i + " "; // On ajoute le nombre à la liste
28     }
29 }
30 alert("Les nombres premiers jusqu'à " + nombre + " sont : " + resultat);
31 </script>
```

### Étape 9 : L'instruction Switch

J'ai vu qu'il existait une alternative plus propre à l'écriture de multiples if...else if : l'instruction switch.

- Elle compare une valeur à plusieurs cas (case).
- Point de vigilance : Le switch utilise une égalité stricte (===). Si je compare le nombre 3 avec la chaîne "3", cela ne fonctionnera pas.
- J'ai aussi appris à regrouper plusieurs case pour exécuter le même code.

exercice réaliser :

Matisse kra  
bts sio

exo 30 :Réécrire le "switch" dans un "if"

Écrivez le code en utilisant `if..else` qui correspondrait au `switch` suivant :

```
1  switch (browser) {
2    case 'Edge':
3      alert( "You've got the Edge!" );
4      break;
5
6    case 'Chrome':
7    case 'Firefox':
8    case 'Safari':
9    case 'Opera':
10     alert( 'Okay we support these browsers too' );
11     break;
12
13   default:
14     alert( 'We hope that this page looks ok!' );
15 }
```

```
1  <script>
2    let browser = prompt("What browser are you using?");
3    if (browser === "Edge") {
4      alert("You've got the Edge!");
5    } else if (browser === "Chrome" || browser === "Firefox" || browser === "Safari" || browser === "Opera") {
6      alert("Okay we support these browsers too");
7    } else {
8      alert("We hope that this page looks ok!");
9    }
10  </script>
```

## exo 31 Réécrire le "if" dans un "switch"

Réécrivez le code ci-dessous en utilisant une seule instruction `switch` :

```
1 let a = +prompt('a?', '');
2
3 if (a == 0) {
4   alert( 0 );
5 }
6 if (a == 1) {
7   alert( 1 );
8 }
9
10 if (a == 2 || a == 3) {
11   alert( '2,3' );
12 }
```

```
1 <script>
2   let a = +prompt('a?' , '');
3
4   switch (a) {
5     case 0:
6       alert(0);
7       break;
8     case 1:
9       alert(1);
10      break;
11     case 2:
12     case 3:
13       alert('2,3');
14       break;
15     default:
16       alert('Other value');
17   }
18 </script>
```

### Étape 10 : Les Fonctions

C'est une partie majeure de l'atelier. J'ai compris que les fonctions sont les "briques" principales d'un programme, permettant de réutiliser du code sans le dupliquer.

- Déclaration et Appel : J'ai créé mes propres fonctions avec le mot-clé `function`, en leur donnant des noms d'actions (verbes comme `showMessage` ou `getAge`).
- Portée des variables (Scope) :
  - Variables locales : Déclarées dans la fonction, elles ne sont pas visibles à l'extérieur.

- Variables globales : Accessibles partout, mais j'ai noté qu'il faut limiter leur usage pour garder un code propre.
- Paramètres et Arguments : J'ai appris à passer des données à mes fonctions. J'ai aussi découvert qu'on peut définir des valeurs par défaut (ex: text = "no text") pour éviter les erreurs si un argument manque.
- Retourner une valeur : Avec return, la fonction renvoie un résultat au code qui l'a appelée. Si on ne met pas de return, la fonction renvoie undefined.

Les différentes syntaxes de fonctions Le cours m'a montré qu'il existe plusieurs façons d'écrire une fonction, chacune ayant ses subtilités :

1. Function Declaration : La méthode classique (function maFonction()). Elle est chargée avant l'exécution du script, ce qui permet de l'appeler avant sa définition dans le code.
2. Function Expression : On stocke la fonction dans une variable (let maFonction = function()). Elle n'est créée que lorsque le code arrive à cette ligne.
3. Fonctions fléchées (Arrow Functions) : Une syntaxe moderne et concise (let func = (arg) => expression) très pratique pour les actions simples sur une ligne.

### exo 31 Est-ce que "else" est requis ?

La fonction suivante renvoie `true` si le paramètre `age` est supérieur à `18`.

Sinon, il demande une confirmation et renvoie son résultat :

```
1 function checkAge(age) {  
2   if (age > 18) {  
3     return true;  
4   } else {  
5     // ...  
6     return confirm('Did parents allow you?');  
7   }  
8 }
```

La fonction fonctionnera-t-elle différemment si `else` est supprimé ?

```
1 function checkAge(age) {  
2   if (age > 18) {  
3     return true;  
4   }  
5   // ...  
6   return confirm('Did parents allow you?');  
7 }
```

Existe-t-il une différence dans le comportement de ces deux variantes ?

Non, il n'y a aucune différence de comportement. Le else n'est pas requis ici.

Cas A : L'utilisateur a 20 ans (age > 18 est VRAI)

- Variante 1 (avec else) : L'ordinateur entre dans le if. Il voit return true. Il s'arrête et sort de la fonction. Le else est ignoré.

Matisse kra  
bts sio

- Variante 2 (sans else) : L'ordinateur entre dans le if. Il voit return true. Il s'arrête et sort de la fonction. La ligne du bas (confirm...) n'est jamais atteinte.
- Résultat : Identique.

Cas B : L'utilisateur a 15 ans (age > 18 est FAUX)

- Variante 1 (avec else) : L'ordinateur saute le if. Il va donc obligatoirement dans le else et exécute return confirm(...).
- Variante 2 (sans else) : L'ordinateur saute le if. Il continue simplement sa lecture vers la ligne suivante, qui est return confirm(...).
- Résultat : Identique.

exo 32 Réécrivez la fonction en utilisant '?' ou '||'

La fonction suivante renvoie true si le paramètre age est supérieur à 18.

Sinon, il demande une confirmation et renvoie le résultat.

```
1 function checkAge(age) {  
2   if (age > 18) {  
3     return true;  
4   } else {  
5     return confirm('Did parents allow you?');  
6   }  
7 }
```

Réécrivez-le, pour effectuer la même chose, mais sans if, et en une seule ligne.

Faites deux variantes de checkAge :

1. En utilisant un opérateur point d'interrogation ?
2. En utilisant OU ||

```
1 <script>  
2 function checkAge(age) {  
3   return (age > 18) ? true : confirm('Did parents allow you?');  
4 }  
5 </script>
```

```
1 <script>  
2 function checkAge(age) {  
3   return (age > 18) || confirm('Did parents allow you?');  
4 }  
5 </script>
```



Matisse kra  
bts sio

### exo 33 Fonction min(a, b)

Ecrivez une fonction `min(a, b)` qui renvoie le plus petit des deux nombres `a` et `b`.

Par exemple :

```
1 min(2, 5) == 2
2 min(3, -1) == -1
3 min(1, 1) == 1
```

```
1 <script>
2 let a = 2;
3 let b = 5;
4
5 function getmin(a, b) {
6   return (a < b) ? a : b;
7 }
8
9
10 alert( getmin(a, b) );
11 </script>
```

### exo 34 Fonction pow(x,n)

Ecrivez une fonction `pow(x, n)` qui renvoie `x` à la puissance `n`. Ou, autrement dit, multiplie `x` par lui-même `n` fois et renvoie le résultat.

```
1 pow(3, 2) = 3 * 3 = 9
2 pow(3, 3) = 3 * 3 * 3 = 27
3 pow(1, 100) = 1 * 1 * ... * 1 = 1
```

Créez une page Web qui demande ( `prompt` ) `x` et `n`, puis affiche le résultat de `pow(x, n)`.

```
1 <script>
2 let n = prompt("Enter a nombre");
3 let x = prompt("entrer la puissance");
4
5 function calcpow(n, x) {
6   return(n ** x);
7 }
8 alert(calcpow(n, x));
9 </script>
```

Matisse kra  
bts sio

## exo 35 Réécrire avec les fonctions fléchées

Remplacez les expressions de fonction par des fonctions fléchées dans le code ci-dessous :

```
1 function ask(question, yes, no) {  
2   if (confirm(question)) yes();  
3   else no();  
4 }  
5  
6 ask(  
7   "Do you agree?",  
8   function() { alert("You agreed."); },  
9   function() { alert("You canceled the execution."); }  
10  );
```

```
1  <script>  
2  const ask = (question, yes, no) => {  
3    if (confirm(question)) yes();  
4    else no();  
5  };  
6  
7  ask(  
8    "Do you agree?",  
9    () => alert("You agreed."),  
10   () => alert("You canceled the execution.")  
11  );  
12  </script>
```

## Conclusion

Cet atelier m'a permis de construire des bases solides en algorithmique avec JavaScript. J'ai dépassé le stade de l'affichage simple pour structurer mon code de manière logique. J'ai appris à maîtriser les flux d'exécution grâce aux boucles et aux conditions avancées (switch). Surtout, j'ai compris l'importance de modulariser mon code grâce aux fonctions. Je sais maintenant faire la différence entre une variable locale et globale, et choisir la syntaxe de fonction la plus adaptée (déclaration classique ou fonction fléchée). Ces compétences sont indispensables pour la suite, notamment pour manipuler le DOM et réagir aux événements utilisateurs de manière complexe.